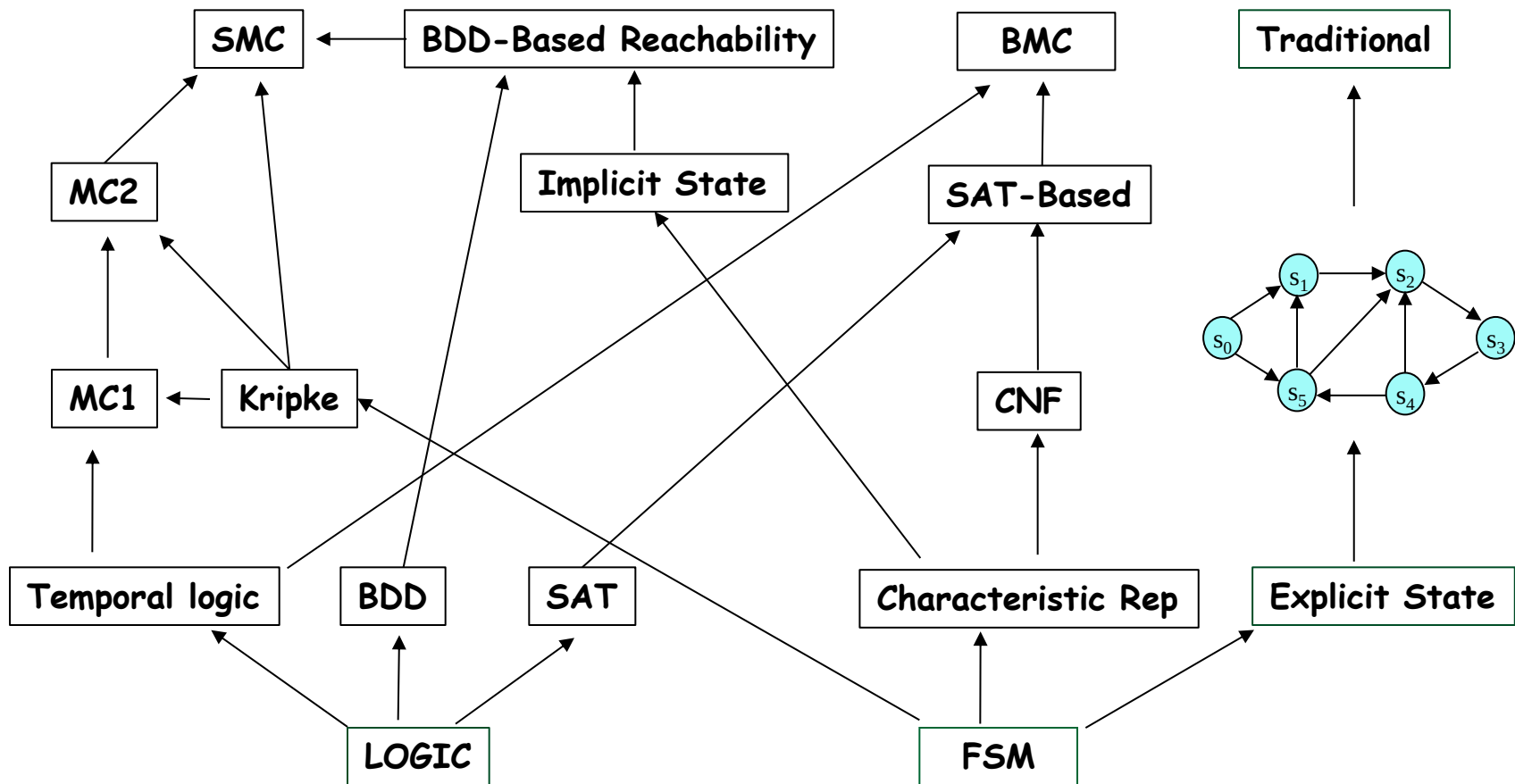


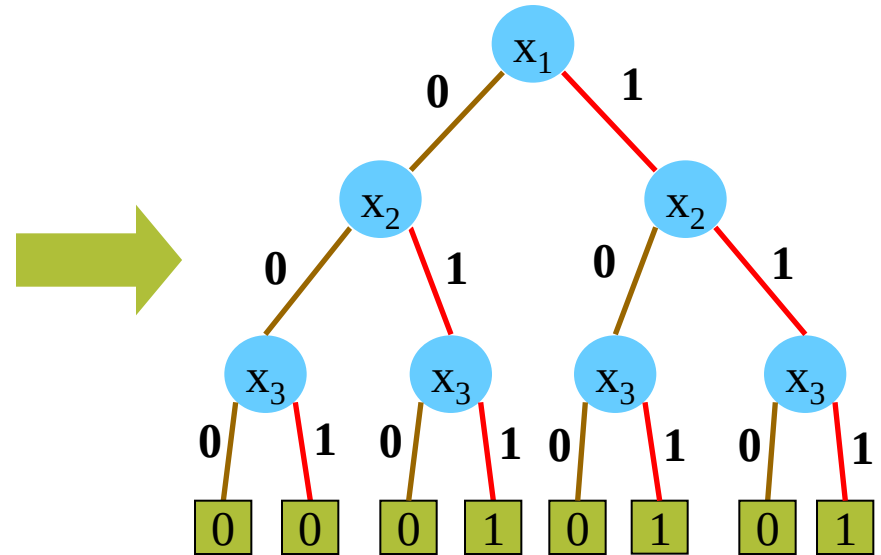
# Binary Decision Diagrams (BDDs)

# A Road Map



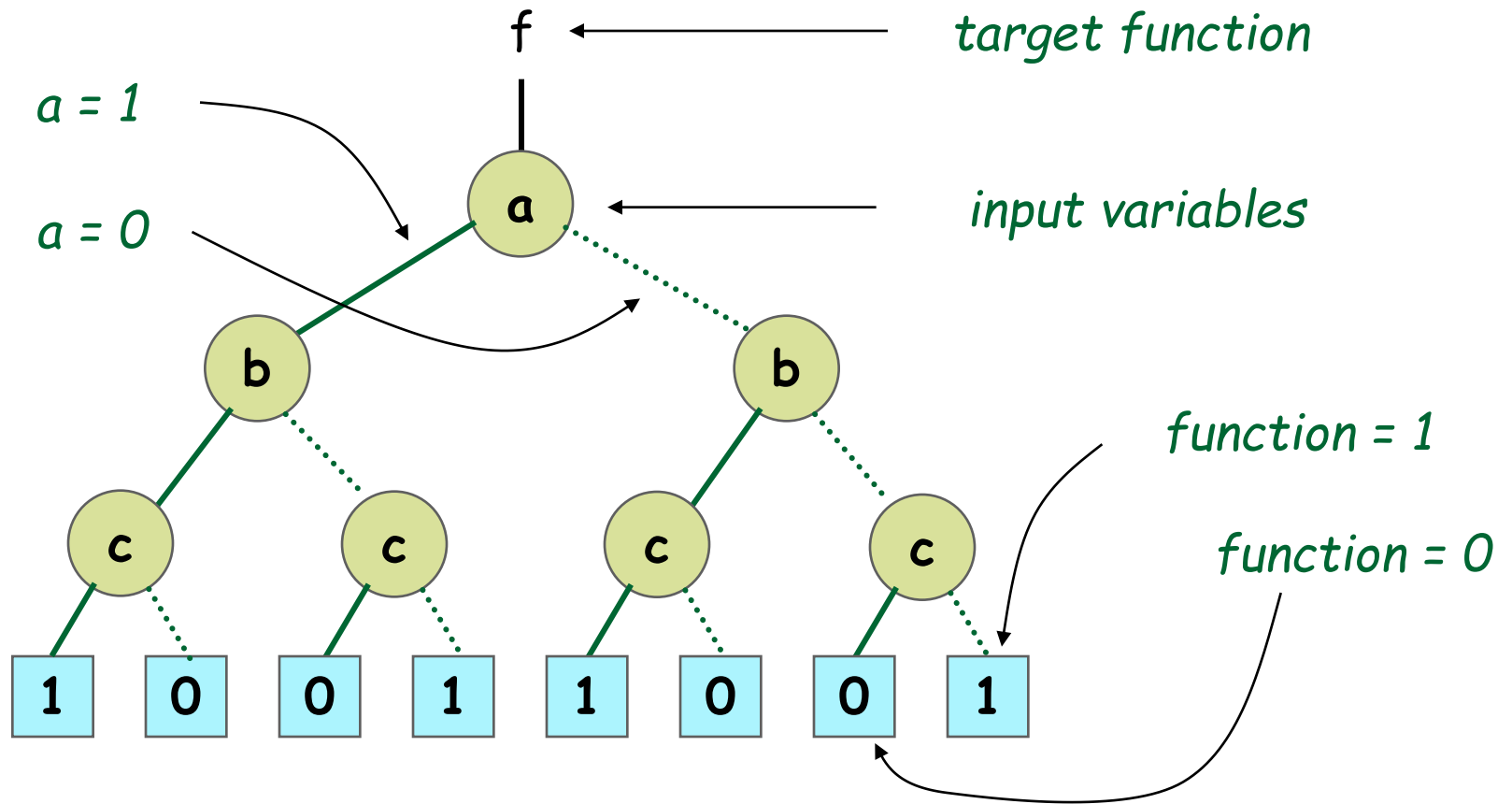
# Truth Table & Decision Tree

|       |   |   |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|---|---|
| $x_1$ | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| $x_2$ | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| $x_3$ | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| F     | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 |



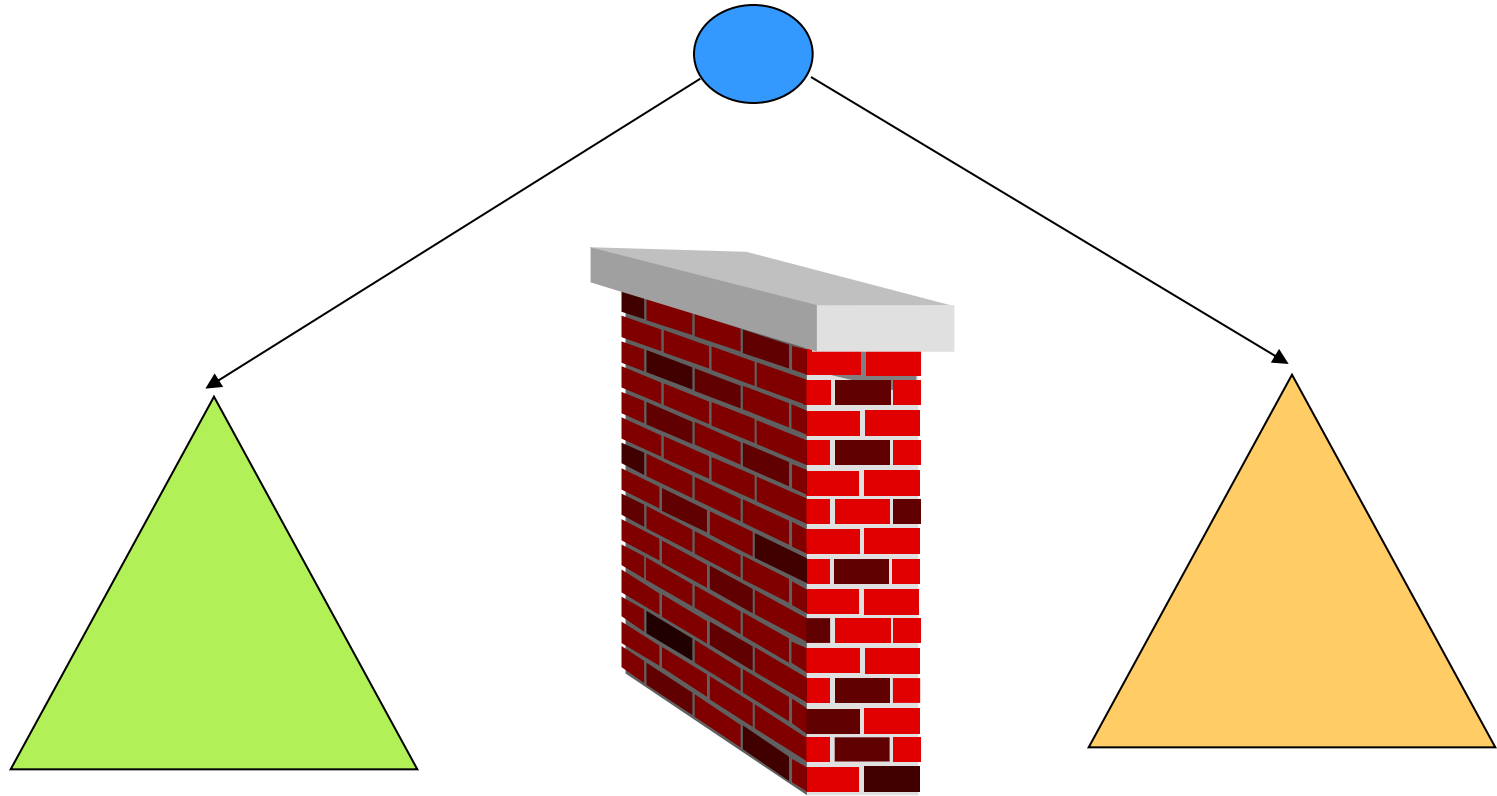
$$f_v(x_1, \dots, x_n) = [ \neg x_i \wedge f_{\text{low}(v)}(x_1, \dots, x_n) ] \vee [ x_i \wedge f_{\text{high}(v)}(x_1, \dots, x_n) ]$$

# Binary Decision Tree

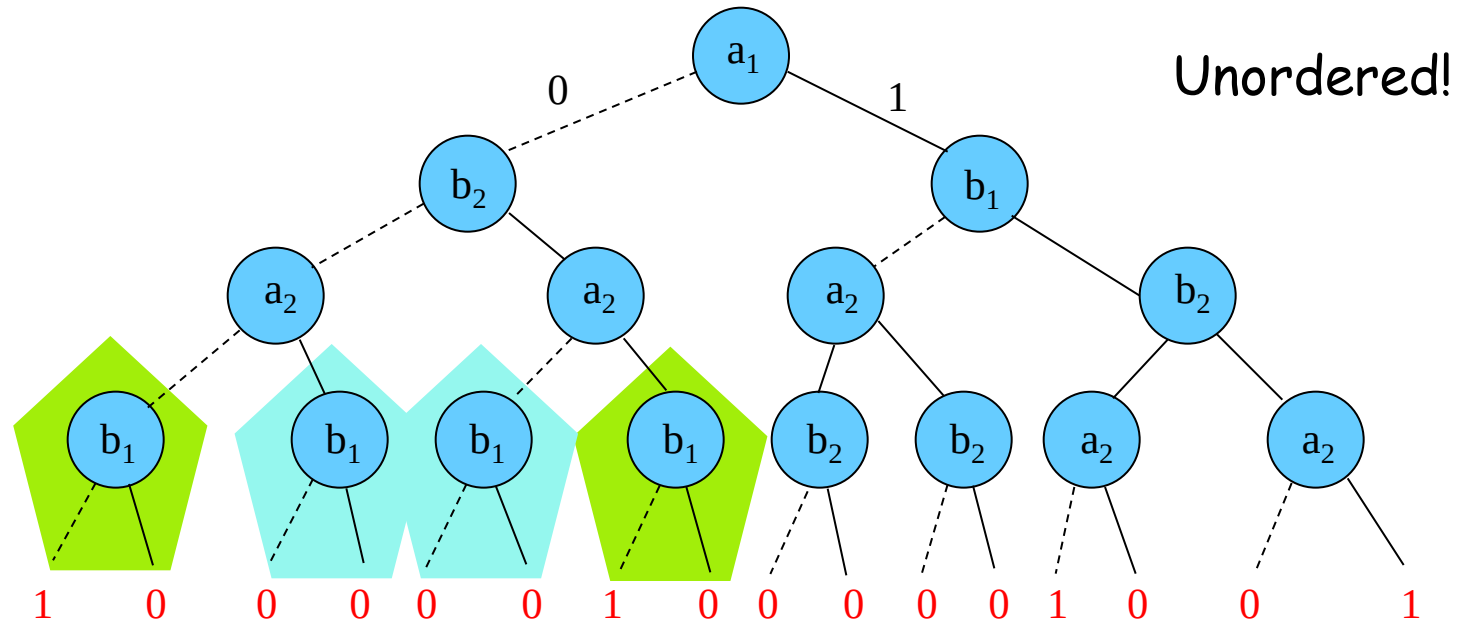


# Binary Decision Tree

$$f_v(x_1, \dots, x_n) = [ \neg x_i \wedge f_{\text{low}(v)}(x_1, \dots, x_n) ] \vee [ x_i \wedge f_{\text{high}(v)}(x_1, \dots, x_n) ]$$



# Binary Decision Trees



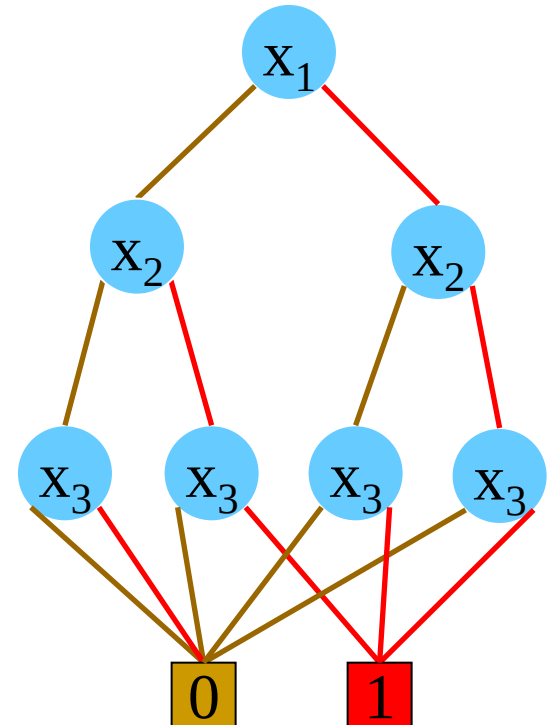
- BDTs are not a concise representation: much redundancy
- A two-bit comparator:  $f(a_1, a_2, b_1, b_2) = (a_1 \Leftrightarrow b_1) \wedge (a_2 \Leftrightarrow b_2)$

# BDD: Binary Decision Diagram (1986)

- Two requirements
  - The variables appear in the same order along each path from the root to a terminal
  - There are no isomorphic subtrees or redundant vertices
- Theorem (Canonicity, Bryant - 1986)
  - Two functions are the same iff their BDD's are equivalent graphs (isomorphic) with same order for variables.

# A Global Ordering

- Impose **total order**  $<$  on the variables.
- Input variables are ordered.

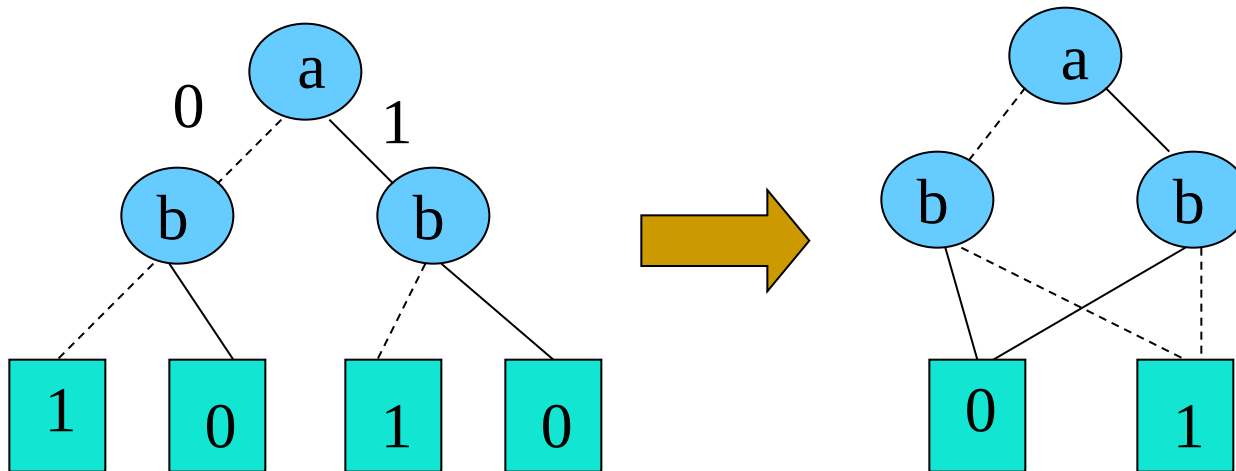




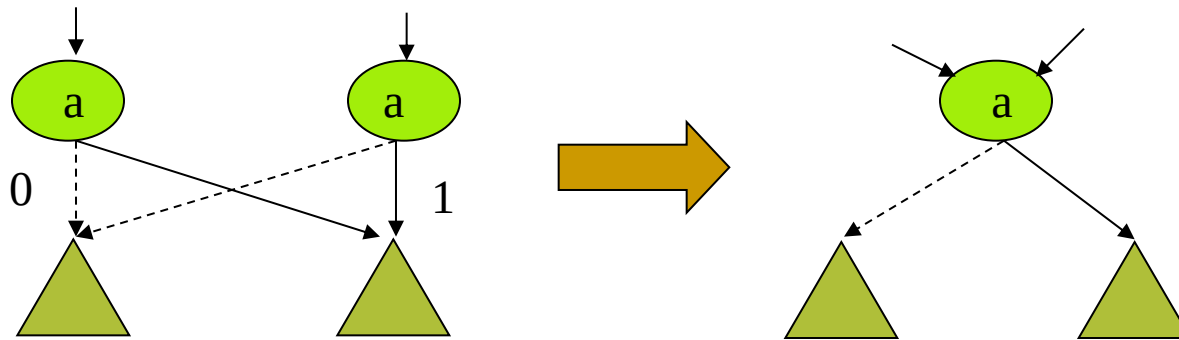
# Reduction Rules

## 1. Remove duplicate terminals

- Eliminate all but one terminal vertex with a given label and redirect all arcs to the eliminated vertices to the remaining one



# Reduction Rules



## 2. Remove duplicate non-terminals (Merging Rule)

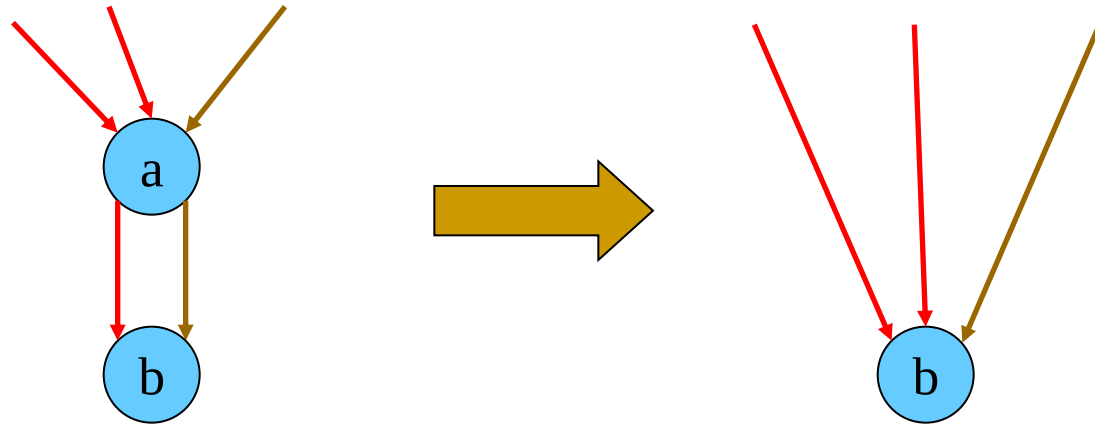
if non-terminals  $u$  and  $v$  have  $\text{var}(u) = \text{var}(v)$ ,

$\text{low}(u) = \text{low}(v)$  and  $\text{high}(u) = \text{high}(v)$ ,

$\Rightarrow$  eliminate one of the two vertices and redirect all incoming arcs to the other vertex

- Nodes must be unique

# Reduction Rules



### 3. Remove redundant tests

if non-terminal vertex  $v$  has  $low(v)=high(v)$ ,

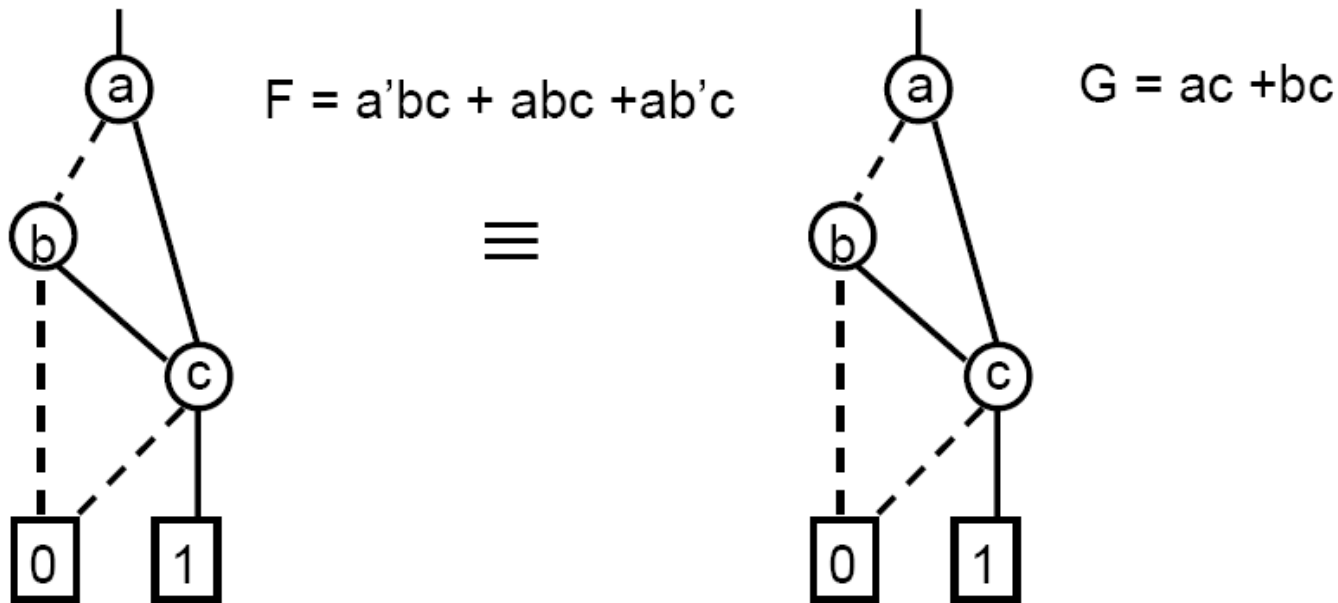
$\Rightarrow$  eliminate  $v$  and redirect all incoming arcs to  $low(v)$

# BDD: Canonical Form

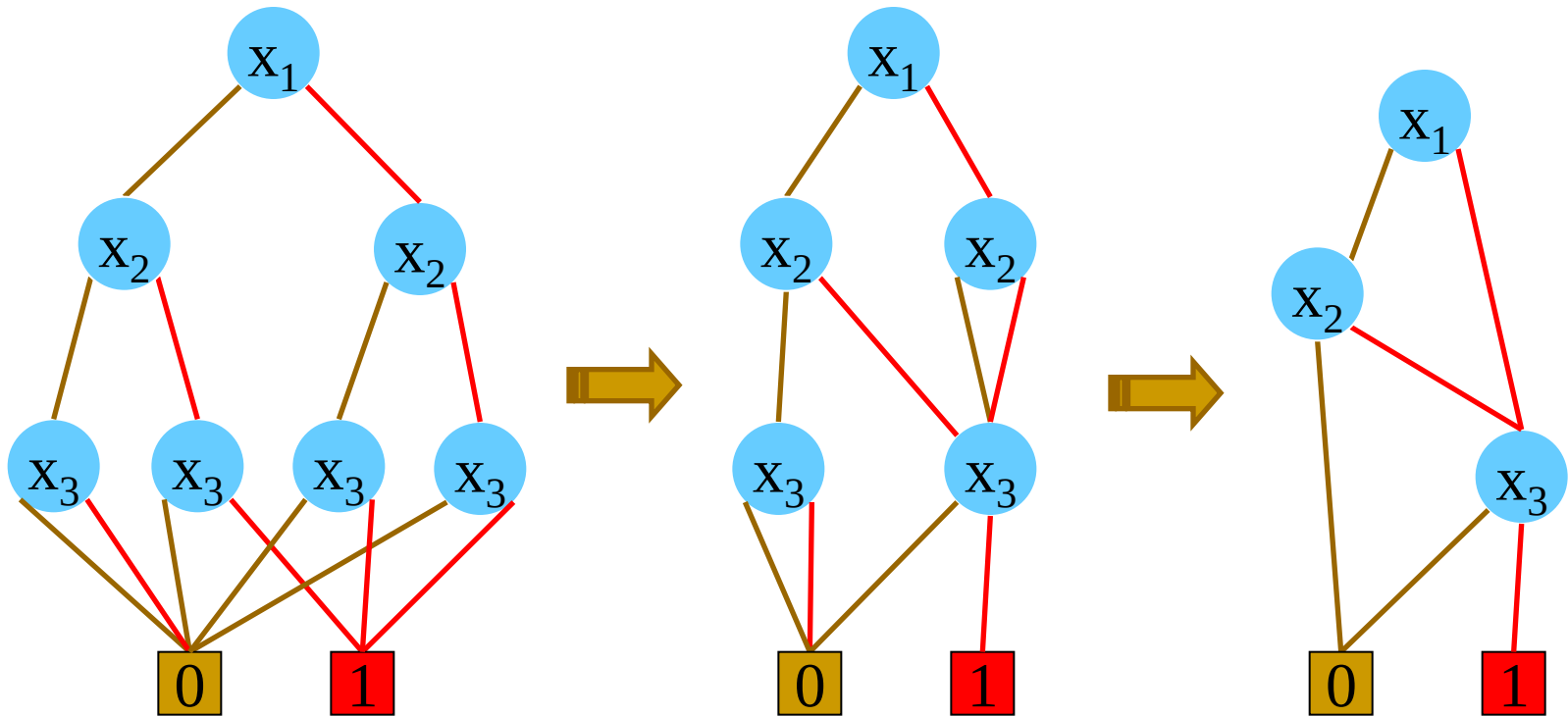
- A canonical form is obtained by
  - applying the transformation rules until none can be further applied

# Canonicity

- If  $F$  and  $G$  are equivalent, their BDDs are identical (for the same ordering of variables)

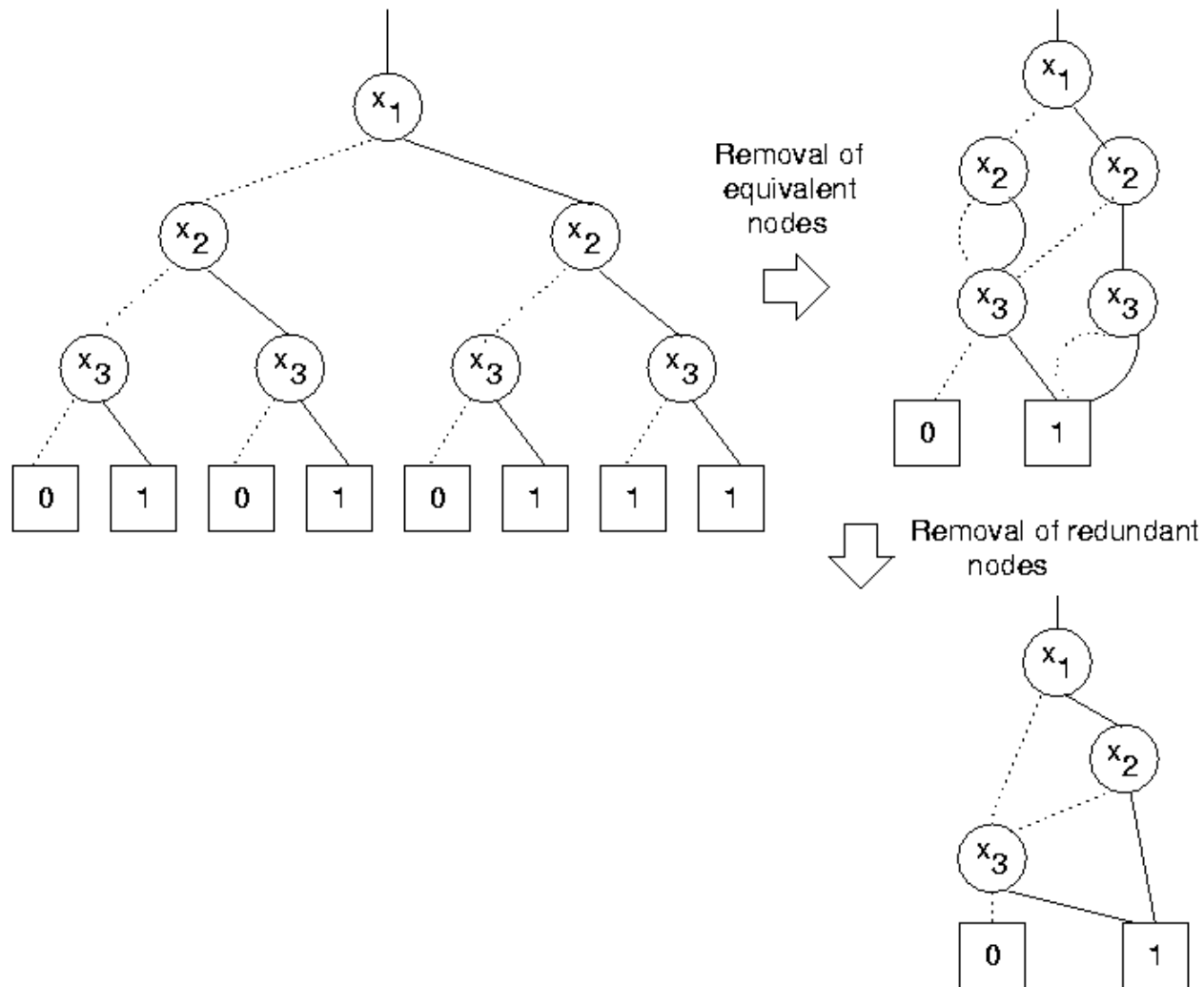


# Building BDDs by Reduction Rules

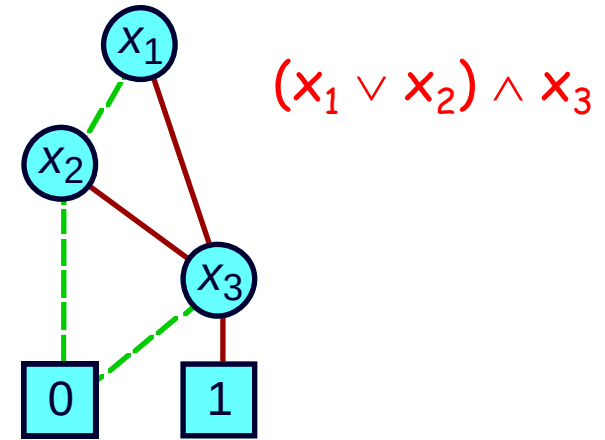
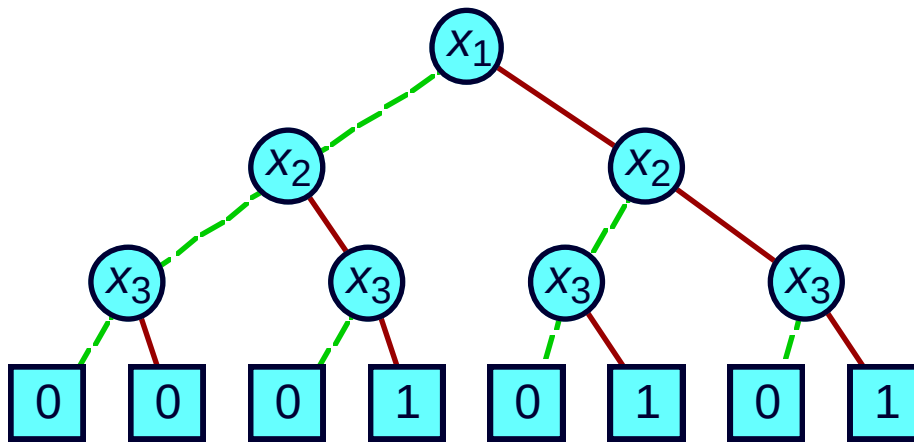


Decision Tree Reduction BDD

# Reduction Rules



# BDD: Canonical Form



- The power of the BDD representation:
  - it does not explicitly *enumerate all paths*,
  - The size is measured by *its nodes and not paths*.
- It can represent an exponential number of paths with a linear number of nodes.



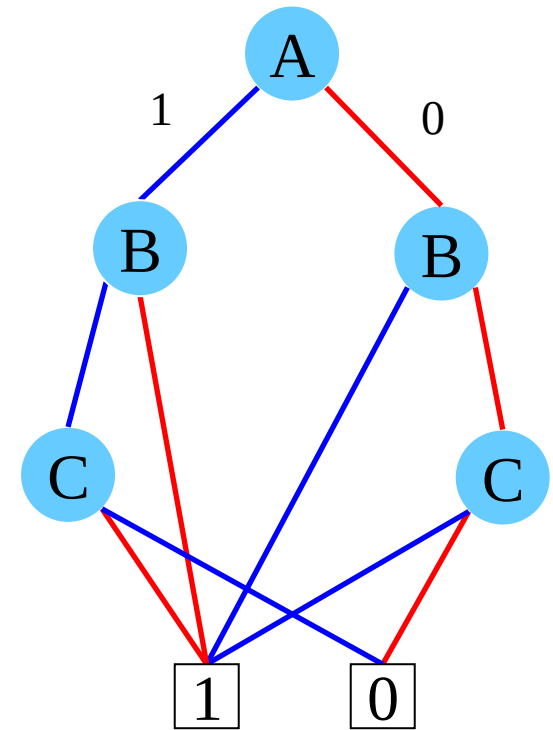
# Not Minimal, But Canonical

|   |   | BA  |     |     |     |
|---|---|-----|-----|-----|-----|
|   |   | 00  | 01  | 11  | 10  |
| C | 0 | 0 0 | 1 1 | 3 1 | 2 1 |
|   | 1 | 4 1 | 5 1 | 7 0 | 6 1 |

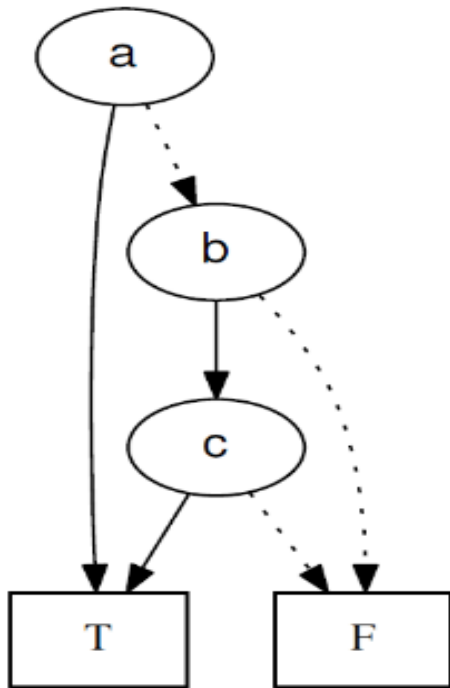
|   |   | BA  |     |     |     |
|---|---|-----|-----|-----|-----|
|   |   | 00  | 01  | 11  | 10  |
| C | 0 | 0 0 | 1 1 | 3 1 | 2 1 |
|   | 1 | 4 1 | 5 1 | 7 0 | 6 1 |

| C \ BA | 00 | 01 | 11 | 10 |
|--------|----|----|----|----|
| 0      | 0  | 1  | 1  | 1  |
| 1      | 1  | 1  | 0  | 1  |

- $F = AB'C' + ABC' + A'B'C' + AB'C + A'BC' + A'BC$
- Two different minimal expressions
  - $AB' + BC' + A'C$
  - $B'C + A'B + AC'$
- BDD reading:
  - $ABC' + AB' + A'B + A'B'C$  (Disjoint covers)
  - $= A(BC' + B') + A'(B + B'C)$  (Not Minimal)



# Not Minimal, But Canonical



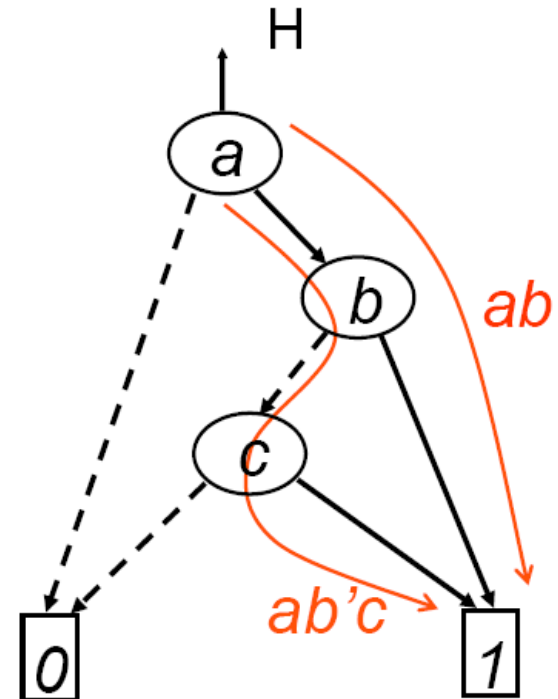
- BDD reading:
  - $F = a + a'bc$  (Disjoint covers, not Minimal)
  - $= a+bc$  (Minimal)

# BDD: Canonical Form

- In many applications, BDDs can be used as an efficient implementation of Boolean representations of the problem.
- For given variable ordering, two functions equivalent if and only if their graphs are isomorphic
  - Test in linear time
- *Equivalence checking:*
  - verify isomorphism between BDDs
- *Satisfiability:*
  - verify only one terminal node labeled by 0

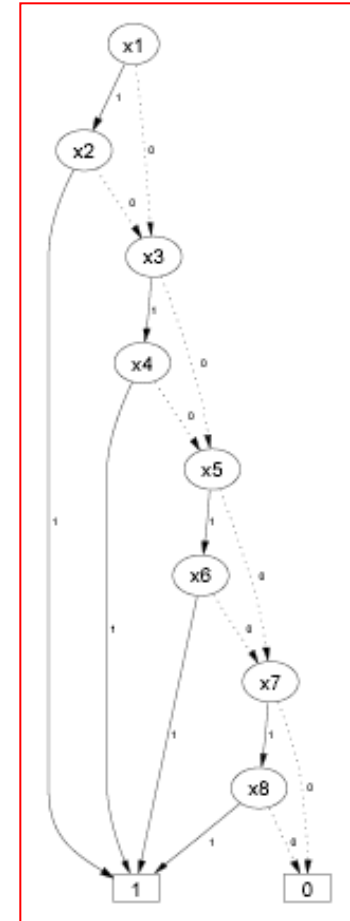
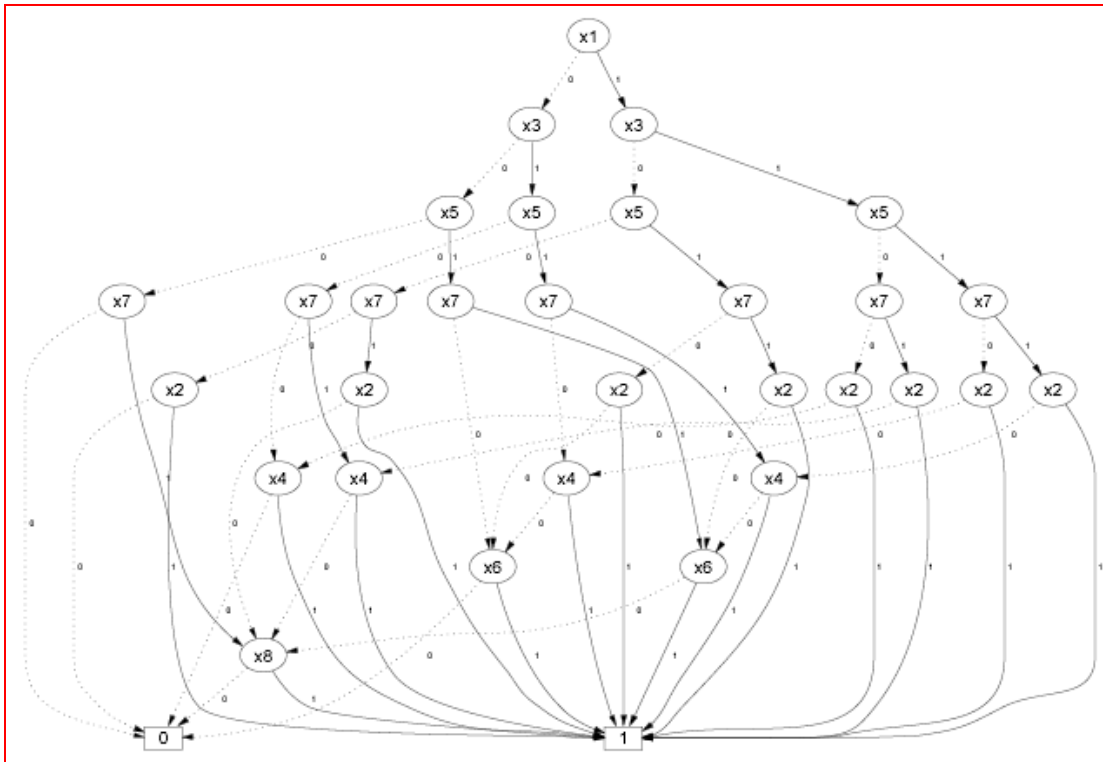
# Other Applications

- Functional test generation
  - SAT, Boolean satisfiability analysis
  - to test for  $H = 1$  (0), find a path in the BDD to terminal 1(0)
  - the path, expressed in function variables, gives a satisfying solution (test vector)



# Variable Ordering

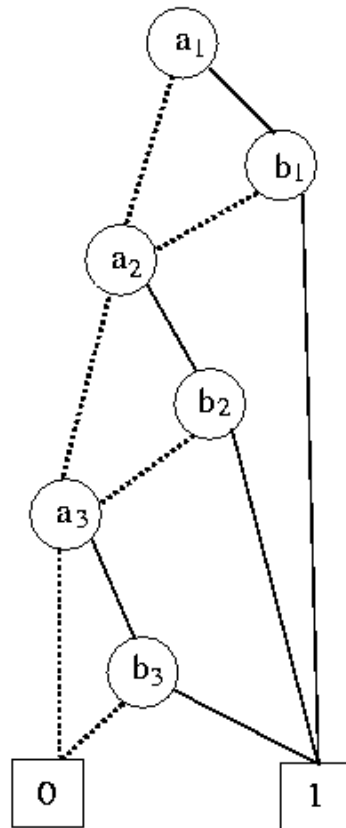
$$F = x_1x_2 + x_3x_4 + x_5x_6 + x_7x_8$$



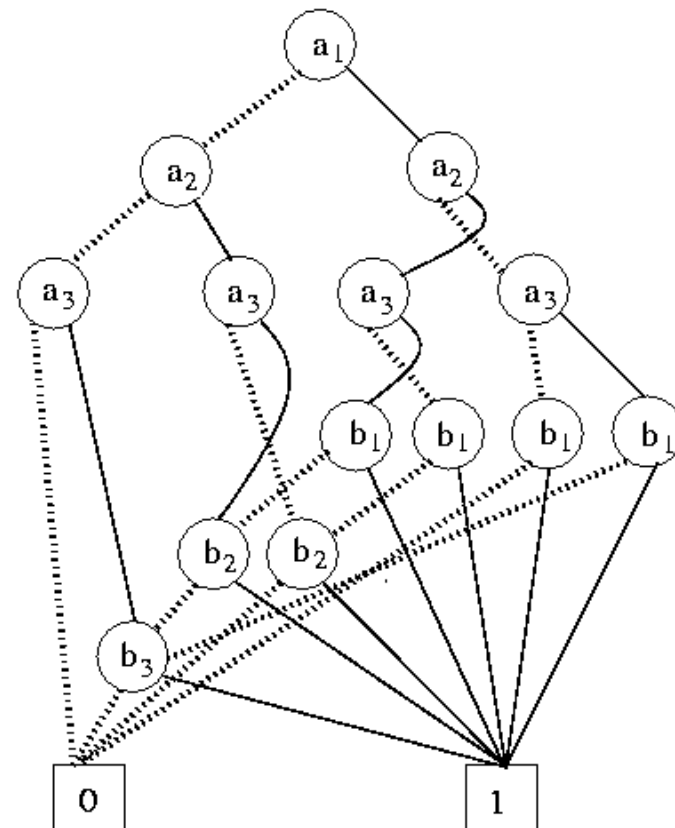
- The size of a BDD depends critically on the variable order.

# Variable Ordering

Function :  $a_1b_1 + a_2b_2 + a_3b_3$



Ordering  $a_1 < b_1 < a_2 < b_2 < a_3 < b_3$

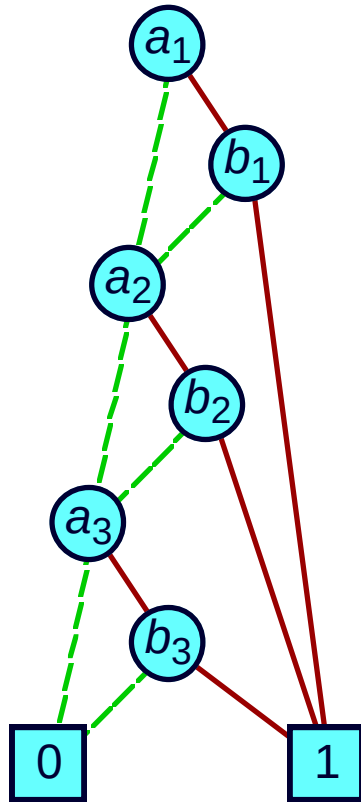


Ordering  $a_1 < a_2 < a_3 < b_1 < b_2 < b_3$

# Effect of Variable Ordering

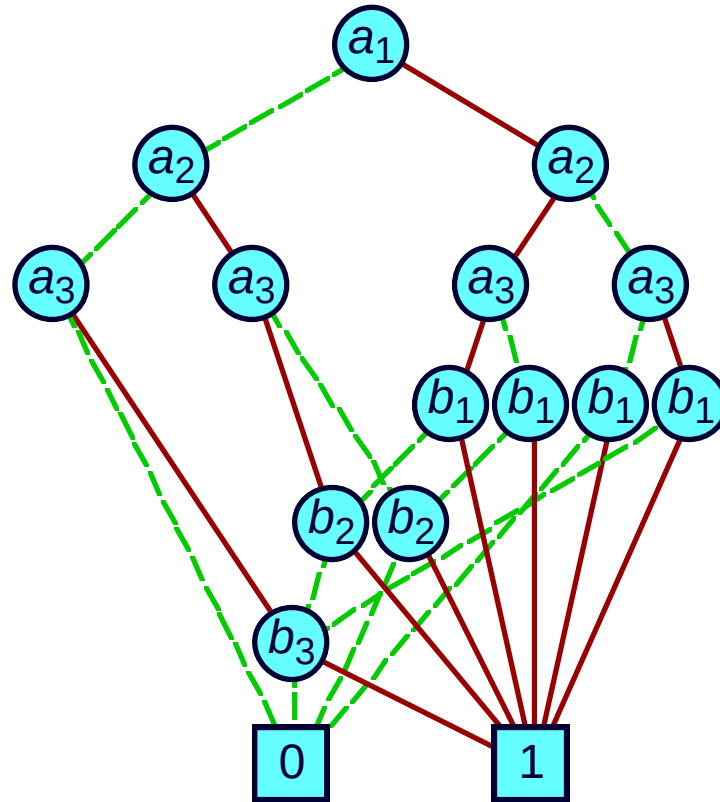
$$(a_1 \wedge b_1) \vee (a_2 \wedge b_2) \vee (a_3 \wedge b_3)$$

## Good Ordering



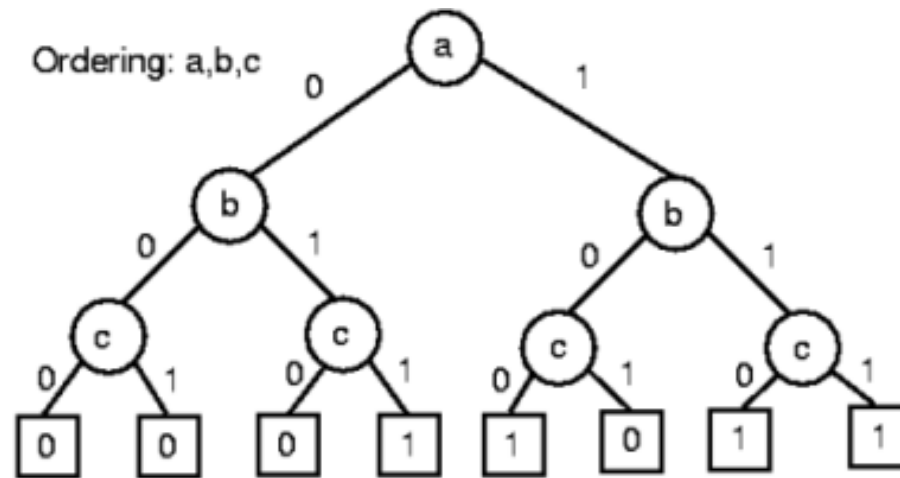
Linear Growth

## Bad Ordering

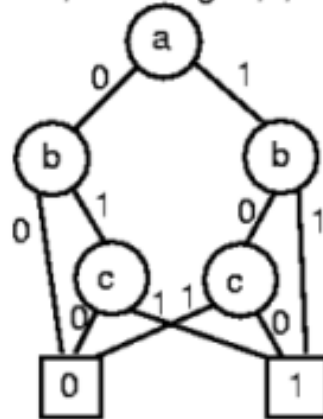


Exponential Growth

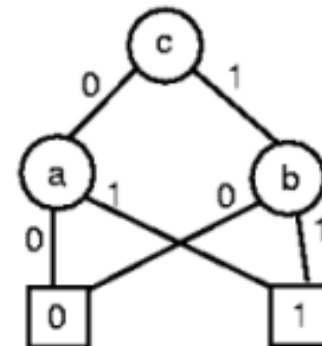
# Variable Ordering



Reduced, Ordering: a,b,c



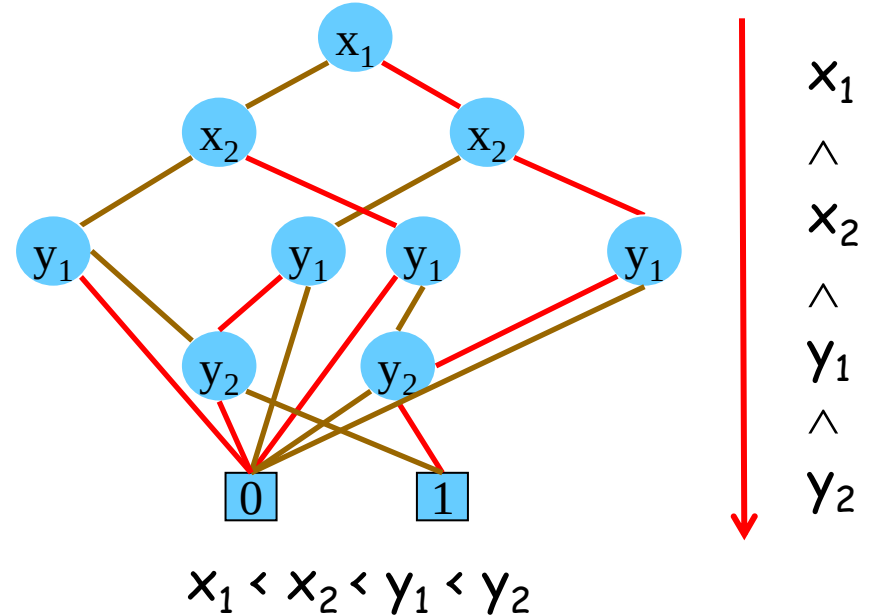
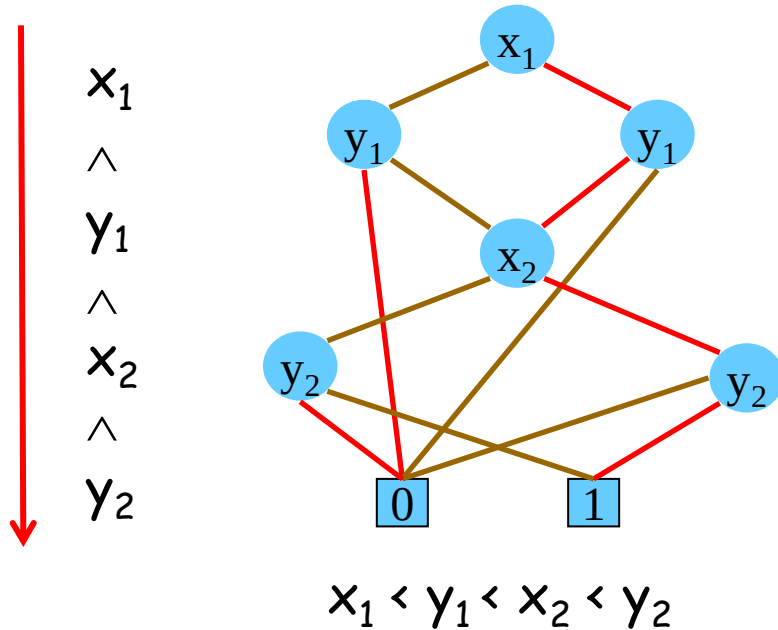
Reduced, Ordering: c,a,b





# Variable Ordering

$$F(x_1, x_2, y_1, y_2) = (x_1 = y_1) \wedge (x_2 = y_2)$$



- For an n-bit comparator:
  - For order  $a_1 < b_1 < \dots < a_n < b_n$ , the number of vertices is  $3n+2$ .
  - For order  $a_1 < \dots < a_n < b_1 \dots < b_n$ , the number of vertices is  $3 \cdot 2^n - 1$ !

# Variable Ordering

- The problem of finding the optimal variable order is NP-complete
- Some Boolean functions have exponential size ROBDDs for any order (e.g., multiplier)

# Heuristics on Variable Ordering

- Heuristics developed for finding a good variable order.
- Intuition for these heuristics:
  - BDDs tend to be smaller when related variables are close together in the order
    - Variables appearing in a subcircuit are related:
    - they determine the subcircuit's output should usually be close together in the order

# References

- Randal Bryant. Graph-Based Algorithms for Boolean Function Manipulation. *IEEE Transactions on Computers*, pp. 677-691, August 1986.
- Randal Bryant. Symbolic Boolean Manipulation with Ordered Binary Decision Diagrams. *ACM Computing Surveys*, 24(3), 1992, pp. 293-318.
- Karl S. Brace, Richard L. Rudell, and Randal E. Bryant. Efficient Implementation of a BDD Package. *27th Design Automation Conference*, pp. 40-45, 1990. (How BDD packages are really implemented).
- M. Huth, M. Ryan, "Logic in Computer Science: Modeling and Reasoning about Systems", Cambridge University Press, 2000.
- C. Meinel and T. Theobald, Algorithms and Data Structures in VLSI Design, Springer.
- R. K. Brayton, Lecture notes on BDD, UC Berkeley.
- H. R. Andersen "An introduction to Binary Decision Diagrams", HRA100497, Technical University of Denmark, Lyngby.
- William K. Lam, "Hardware Design Verification: Simulation and Formal Method-Based Approaches", Prentice Hall, 2005.